

Pytania kierunkowe Informatyka IST:

1. Zdefiniować pojęcie algorytmu i opisać jego własności

Skończony i jednoznaczny ciąg krok po kroku instrukcji, które prowadzi do rozwiązania określonego problemu.

Jego głównymi cechami są:

- **Skończoność**: musi zakończyć się w skończonej ilości kroków
- **Jednoznaczność**: krok musi być precyzyjnie określony tak by te same dane dawały zawsze ten sam wynik
- **Poprawność oraz uniwersalność**: musi poprawnie rozwiązywać zadanie i może być stosowany do rozwiązywania wielu problemów tego samego typu

2. Wyjaśnić czym są i jaką rolę pełnią w programie instrukcje sterujące

Instrukcje zdefiniowane w składni określonego języka programowania służące do kontrolowania przepływu działania programu. Pozwalają na wykonanie kodu źródłowego sekwencyjne, iteracyjne, warunkowe, lub w sposób nieciągły (goto, break, continue, return)

3. Scharakteryzować główne rodzaje cyfrowych bloków funkcjonalnych: bloki komutacyjne, bloki arytmetyczne, rejestry, liczniki. Pamięci

Bloki komutacyjne (kombinacyjne) – wyjście zależy tylko od aktualnych wejść, brak pamięci stanu. Realizują funkcje logiczne i wybór sygnałów.

Bloki arytmetyczne – wykonują operacje matematyczne na danych binarnych (dodawanie, odejmowanie itd.).

Rejestry – przechowują dane binarne, zbudowane z przerzutników, działają synchronicznie z zegarem.

Liczniki – specjalne rejestry zliczające impulsy zegarowe według określonej sekwencji.

Pamięci – służą do przechowywania danych; umożliwiają zapis i odczyt informacji (ulotne i nieulotne).

4. Omówić podstawowe zasady projektowania i realizacji synchronicznych cyfrowych układów sekwencyjnych

Synchroniczne układy sekwencyjne działają w takt wspólnego sygnału zegarowego. Podstawowe zasady ich projektowania:

- **Wspólny zegar** – wszystkie elementy pamiętające (przerzutniki, rejestry) są sterowane tym samym sygnałem zegara.
- **Stany zmieniają się tylko na zboczu zegara** – zapewnia to przewidywalność działania układu.
- **Rozdzielenie logiki** – logika kombinacyjna oblicza następny stan, a elementy sekwencyjne go przechowują.
- **Spełnienie czasów** – należy zachować czasy propagacji, setup i hold, aby uniknąć błędów.

- **Reset/ustawienie początkowe** – układ powinien mieć zdefiniowany stan startowy.
- **Unikanie hazardów i wyścigów** – stosowanie synchronizacji i odpowiedniego taktowania.

5. Scharakteryzować systemy liczbowe NKB i U2 dla liczb całkowitych o ograniczonej długości do n cyfr (bitów)

NKB (Naturalny Kod Binarny)

- Reprezentuje **liczby nieujemne**.
- Zakres: od 000 do $2^n - 1$.
- Wszystkie bity opisują wartość liczby.
- Prosty w interpretacji, brak liczb ujemnych.

U2 (kod uzupełnień do dwóch)

- Reprezentuje **liczby dodatnie i ujemne**.
- Najstarszy bit jest bitem znaku.
- Zakres: od -2^{n-1} do $2^{n-1} - 1$.
- Umożliwia proste wykonywanie operacji arytmetycznych (to samo dodawanie dla $+$ i $-$).

6. Scharakteryzować abstrakcyjne typy danych (rodzaje, własności, wady i zalety).

Abstrakcyjne typy danych (ADT) – opisują **dane przez dozwolone operacje**, a nie przez sposób implementacji.

Główne rodzaje:

- **Stos (LIFO)** – dostęp do ostatniego elementu.
 - prostota, szybkie operacje
 - brak dostępu losowego
- **Kolejka (FIFO)** – dostęp do pierwszego elementu.
 - naturalna obsługa kolejek zdarzeń
 - ograniczony dostęp do danych
- **Lista** – uporządkowana sekwencja elementów.
 - elastyczność, łatwe wstawianie/usuwanie
 - wolniejszy dostęp niż w tablicach
- **Zbiór** – brak duplikatów.
 - szybkie sprawdzanie przynależności
 - brak kolejności
- **Mapa (słownik)** – pary klucz–wartość.

- szybki dostęp po kluczu
 - większe zużycie pamięci

Własności ADT:

- niezależność od implementacji,
- jasno zdefiniowany interfejs,
- enkapsulacja danych.

Zalety:

- łatwiejsze projektowanie i modyfikacja kodu,
- czytelność i bezpieczeństwo,
- możliwość zmiany implementacji bez zmiany interfejsu.

Wady:

- narzut wydajnościowy,
- mniejsza kontrola nad szczegółami implementacji.

7. Na wybranym przykładzie omówić metodę dziel i zwyciężaj konstruowania algorytmów

Dziel i zwyciężaj – metoda polegająca na **podziale problemu na mniejsze podproblemy**, ich rozwiązaniu i połączeniu wyników.

Przykład: sortowanie przez scalanie (Merge Sort)

1. **Dziel** – tablicę dzieli się na dwie mniejsze części.
2. **Zwyciężaj** – każdą część sortuje się rekurencyjnie.
3. **Scal** – połączenie dwóch posortowanych tablic w jedną.

Cechy metody:

- rekurencyjna struktura,
- wysoka czytelność algorytmu,
- dobra wydajność (Merge Sort: $O(n \log n)$).

Zalety:

- łatwe rozwiązywanie złożonych problemów,
- często lepsza złożoność czasowa.

Wady:

- narzut pamięci (rekursja, dodatkowe struktury),
- czasem większa złożoność implementacji.

8. Wyjaśnić pojęcia klucz publiczny oraz klucz prywatny w kontekście algorytmów szyfrujących

Klucz publiczny – klucz **jawny**, udostępniany innym. Służy do **szyfrowania danych** lub **weryfikacji podpisu**.

Klucz prywatny – klucz **tajny**, znany tylko właścicielowi. Służy do **deszyfrowania danych** lub **tworzenia podpisu cyfrowego**.

Dane zaszyfrowane kluczem publicznym można odszyfrować **tylko odpowiadającym mu kluczem prywatnym**.

Przykład: algorytm RSA

- Użytkownik tworzy **parę kluczy**: publiczny i prywatny.
- **Klucz publiczny** udostępnia wszystkim.

Szyfrowanie:

Ktoś szyfruje wiadomość **kluczem publicznym** odbiorcy → tylko odbiorca może ją odczytać.

Deszyfrowanie:

Odbiorca odszyfrowuje wiadomość **kluczem prywatnym**.

9. Omówić pojęcia: rozkaz, cykl rozkazowy CPU, mikroinstrukcja, mikroprogramowanie

- **Rozkaz** – pojedyncza instrukcja procesora określająca operację do wykonania (np. dodawanie, przesunięcie).
- **Cykl rozkazowy CPU** – sekwencja etapów wykonania rozkazu: **pobranie (fetch)**, **dekodowanie (decode)**, **wykonanie (execute)**, **zapis wyniku (write back)**.
- **Mikroinstrukcja** – najmniejsza jednostka sterowania w mikroprogramie, określająca sygnały dla poszczególnych elementów CPU w danym cyklu.
- **Mikroprogramowanie** – sposób realizacji rozkazów procesora poprzez **ciąg mikroinstrukcji**, zamiast bezpośredniego sprzętowego sterowania.

10. Omówić zagadnienie pamięci podręcznej CPU

Pamięć podręczna CPU (cache) to szybka pamięć używana przez procesor do przechowywania **najczęściej używanych danych i instrukcji** z pamięci głównej, aby zmniejszyć czas dostępu.

Działanie:

1. Procesor sprawdza najpierw cache, czy potrzebne dane tam są (**hit**).
2. Jeśli nie (**miss**), dane pobierane są z pamięci głównej i zapisywane w cache na przyszłość.

Struktura:

- **L1** – bardzo szybka, mała, bezpośrednio w CPU, dla instrukcji i danych osobno,
- **L2** – większa i wolniejsza, także w CPU lub blisko niego,
- **L3** – wspólna dla rdzeni, większa, wolniejsza niż L2.

Korzyści:

- znacząco przyspiesza działanie CPU,

- zmniejsza liczbę opóźnień przy odczycie danych z RAM.

Ograniczenia:

- ograniczona pojemność,
- kosztowna w produkcji.

11. Omówić prawdopodobieństwo zupełne i metodę Bayesa

Prawdopodobieństwo zupełne:

„Prawdopodobieństwo zdarzenia A to suma prawdopodobieństw A w każdym możliwym przypadku, pomnożonych przez prawdopodobieństwo tych przypadków.”

$$P(A) = \sum_i P(A|B_i) \cdot P(B_i)$$

Wzór Bayesa:

„Prawdopodobieństwo, że zdarzenie B_i zaszło, jeśli zaszło A, to prawdopodobieństwo A przy B_i razy prawdopodobieństwo B_i , podzielone przez sumę takich iloczynów dla wszystkich możliwych przypadków.”

$$P(B_i|A) = \frac{P(A|B_i) \cdot P(B_i)}{\sum_j P(A|B_j) \cdot P(B_j)}$$

12. Omówić rozkład normalny ciągłych zmiennych losowych

Rozkład normalny (Gaussa) opisuje **ciągłą zmienną losową**, której wartości są skupione wokół średniej i mają charakterystyczny „dzwonowaty” kształt.

Cechy rozkładu normalnego:

- **Symetryczny** względem średniej μ .
- Większość wartości leży w pobliżu μ , odległości mierzy odchylenie standardowe σ .
- Gęstość prawdopodobieństwa:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Wartość oczekiwana = μ , wariancja = σ^2 .

Rozkład normalny opisuje dane skupione wokół średniej i ma kształt dzwonu, symetryczny po obu stronach.

Większość wartości jest blisko średniej: 68% w jednym odchyleniu standardowym, 95% w dwóch, 99,7% w trzech.

Odchylenie standardowe pokazuje rozrzut danych.

Stosuje się go w pomiarach, wynikach testów i przy błędach pomiarowych.”

13. Wyjaśnić czym jest i jaką rolę pełni w programie zmienna wskaźnikowa (wskaźnikowa)

Zmienna wskaźnikowa to zmienna, która przechowuje **adres innej zmiennej** w pamięci.

Jej rola w programie:

- Pozwala na **bezpośredni dostęp do pamięci**.
- Umożliwia **dynamiczne zarządzanie pamięcią**.
- Ułatwia **przekazywanie dużych struktur danych do funkcji** bez kopiowania.
- Jest podstawą **struktur danych** jak listy, drzewa czy tablice dynamiczne.

14. Wyjaśnić czym są funkcje rekurencyjne, podać ich wady i zalety oraz przedstawić przykładowe zastosowania

Funkcje rekurencyjne to funkcje, które **wywołują same siebie** w celu rozwiązania problemu. Zawsze muszą mieć **warunek zakończenia**, aby uniknąć nieskończonej pętli.

Zalety:

- Czystszy, bardziej przejrzysty kod dla problemów naturalnie rekurencyjnych (np. drzewa, grafy).
- Łatwiejsze rozwiązywanie problemów dziel i zwyciężaj.

Wady:

- Większe zużycie pamięci (sterta i stos wywołań).
- Może prowadzić do przepełnienia stosu przy zbyt głębokiej rekurencji.
- Czasami wolniejsze niż iteracja.

Przykłady zastosowań:

- Obliczanie silni lub ciągu Fibonacciego.
- Przeszukiwanie drzew i grafów (DFS).
- Algorytmy dziel i zwyciężaj (np. sortowanie szybkie, sortowanie przez scalanie).
- Rozwiązywanie problemów kombinatorycznych (np. wieże Hanoi, generowanie permutacji).

15. Przedstawić i porównać pojęcia procesu sekwencyjnego i wątku

Proces sekwencyjny:

- To samodzielny program wykonywany przez system operacyjny.
- Posiada **własną przestrzeń adresową** i zasoby (pamięć, pliki).
- Wykonuje instrukcje **po kolei**, jedna po drugiej.

Wątek (thread):

- To lekki podproces działający w ramach procesu.
- **Dzieli zasoby procesu**, np. pamięć i otwarte pliki, ale ma własny stos i rejestry.
- Może działać **równolegle** z innymi wątkami tego samego procesu.

Cecha	Proces sekwencyjny	Wątek
Przestrzeń pamięci	Własna	Dzielona z innymi wątkami
Koszt tworzenia	Wysoki	Niski
Komunikacja	Przez IPC (kosztowna)	Bezpośrednia (wspólna pamięć)
Równoległość	Niezależna	Współbieżna w tym samym procesie
Odporność na błędy	Błędy w procesie nie wpływają na inne	Błędy mogą uszkodzić cały proces

16. Wyjaśnić pojęcia: semafor, mutex, sekcja krytyczna, zdarzenie

Semafor:

- Mechanizm synchronizacji wątków/procesów.
- Przechowuje licznik, który kontroluje dostęp do zasobów współdzielonych.
- Może być **binarny** (0 lub 1) lub liczbowy (większy niż 1).

Mutex (mutual exclusion):

- Specjalny typ semafora binarnego.
- Zapewnia, że **tylko jeden wątek** może wejść do sekcji krytycznej w danym czasie.

Sekcja krytyczna:

- Fragment kodu, który **musi być wykonywany tylko przez jeden wątek** na raz, aby uniknąć konfliktów przy współdzielonych zasobach.

Zdarzenie (event):

- Mechanizm synchronizacji wątków, pozwalający **oczekiwać na wystąpienie pewnego stanu lub zdarzenia**.
- Wątek może być uśpiony do momentu, gdy zdarzenie zostanie „wywołane” przez inny wątek lub proces.

17. Wymienić oraz scharakteryzować protokoły warstwy transportowej modelu ISO/OSI

W warstwie transportowej modelu ISO/OSI działają głównie dwa protokoły:

1. TCP (Transmission Control Protocol)

- **Charakterystyka:** połączeniowy protokół zapewniający niezawodną transmisję danych.
- **Funkcje:** dzielenie danych na segmenty, numerowanie, potwierdzenia odbioru, retransmisja utraconych segmentów, kontrola przepływu i kolejności.
- **Zastosowanie:** przeglądarki internetowe (HTTP/HTTPS), e-mail (SMTP, IMAP), transfer plików (FTP).

2. UDP (User Datagram Protocol)

- **Charakterystyka:** bezpołączeniowy protokół, szybki, ale **nie gwarantuje dostarczenia ani kolejności danych**.
- **Funkcje:** wysyła datagramy bez potwierdzeń i retransmisji, niska latencja.
- **Zastosowanie:** transmisje strumieniowe audio/wideo, gry online, DNS.

Cecha	TCP	UDP
Połączeniowy	Tak	Nie
Niezawodność	Tak	Nie
Kontrola błędów	Tak	Ograniczona
Szybkość	Wolniejszy	Szybszy
Kolejność danych	Zachowana	Nie gwarantowana

18. Scharakteryzować rodzaje mediów transmisyjnych stosowanych w sieciach komputerowych.

Kable miedziane

- **Skłętka (UTP/STP):** tania, łatwa w instalacji, stosowana w LAN; ograniczona prędkość i zasięg.
- **Kabel koncentryczny:** dawniej używany w sieciach Ethernet, odporny na zakłócenia, większy zasięg niż skłętka.

Światłowody (Fiber optic)

- Transmisja światłem, bardzo szybka i na duże odległości.
- Odporny na zakłócenia elektromagnetyczne, wysoka przepustowość.
- Droższy w instalacji i naprawie.

Media bezprzewodowe (Wireless)

- **Wi-Fi, Bluetooth, LTE/5G:** transmisja fal radiowych lub mikrofal.
- Wygodne w instalacji, mobilne, ale podatne na zakłócenia i mają ograniczony zasięg.

Transmisja satelitarna: duży zasięg globalny, opóźnienia w transmisji.

Transmisja kablowa (np. TV kablowa jako medium internetowe): wykorzystuje istniejącą infrastrukturę kablową.

19. Omówić wybraną metodą przybliżania funkcji wielomianami

Interpolacja Newtona to metoda przybliżania funkcji wielomianem,

Dla punktów $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ wielomian interpolacyjny Newtona zapisuje się w postaci:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0)(x - x_1)\dots(x - x_{n-1})$$

Opis zmiennych:

- x – argument, dla którego liczymy wartość wielomianu.
- $x_0, x_1, \dots, x_n$ – współrzędne punktów danych.
- $y_0, y_1, \dots, y_n$ – wartości funkcji w punktach danych.
- $a_0, a_1, \dots, a_n$ – współczynniki Newtona obliczane za pomocą **różnic dzielonych**.

Krok 1: różnice pierwszego rzędu

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{3 - 2}{2 - 1} = 1$$

$$f[x_1, x_2] = \frac{f(x_2) - f(x_1)}{x_2 - x_1} = \frac{5 - 3}{4 - 2} = 1$$

Krok 2: różnice drugiego rzędu

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{1 - 1}{4 - 1} = 0$$

Krok 3: zapis wielomianu Newtona

Wzór wielomianu Newtona:

$$P_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1)$$

20. Omówić całkowanie numeryczne wraz z wybraną metodą

Całkowanie numeryczne służy do **przybliżonego obliczania całek**, gdy nie da się ich obliczyć analitycznie lub funkcja jest znana tylko w punktach.

Najprostsza metoda to **metoda prostokątów (Metoda prostokątów lewych lub prawych)**:

Zasada działania:

- Dzielimy przedział $[a, b]$ na n równych podprzedziałów o szerokości $\Delta x = \frac{b-a}{n}$.
- Wartość całki przybliżamy sumą pól prostokątów:

Dla metody prostokątów lewych:

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} f(x_i) \Delta x$$

Dla metody prostokątów prawych:

$$\int_a^b f(x) dx \approx \sum_{i=1}^n f(x_i) \Delta x$$

Opis zmiennych:

- a, b – granice całkowania.
- n – liczba podprzedziałów.
- Δx – szerokość podprzedziału.
- x_i – punkt w podprzedziale (lewy lub prawy koniec).

Zalety:

- Bardzo prosta do implementacji.
- Działa nawet gdy funkcja jest znana tylko w punktach.

Wady:

- Mała dokładność dla funkcji mocno zmiennych.
- Aby uzyskać lepszą dokładność, trzeba zwiększyć liczbę podprzedziałów.

Pytania specjalnościowe Informatyka IST Teleinformatyka:

21. Omówić metodę oceny niepewności pomiaru

Metoda oceny niepewności pomiaru polega na oszacowaniu przedziału, w którym z określonym prawdopodobieństwem znajduje się wartość prawdziwa wyniku.

Niepewność typu A – wyciągana za pomocą metod statystycznych:

Wykonujemy n pomiarów a następnie wykonujemy obliczenie wartości średniej.
Liczmy sumę kwadratów odchyleń od średniej ze wszystkich wyników:

$$\sum (x_i - \bar{x})^2$$

Obliczamy odchylenie standardowe próby

$$s = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$$

A następnie podstawiamy do wzoru na niepewność średniej

$$u_A = \frac{s}{\sqrt{n}}$$

Interpretujemy jako niepewność u_A w serii pomiarów wynik $\pm u_A$

Niepewność typu B – oparty na wiedzy i danych zewnętrznych (świadczenia kalibracji, dane producenta, normy i dokumentacje)

- Źródło: dane producenta, certyfikaty, normy, doświadczenie – **nie z pomiarów**.
- Sprowadza się do odchylenia standardowego (u_B).
- Dla rozkładu prostokątnego:

$$u_B = \frac{a}{\sqrt{3}}$$

gdzie (a) – granica błędu przyrządu.

- W praktyce łączy się z niepewnością typu A, by uzyskać **niepewność złożoną**.

- Służy do oceny wpływu błędów systematycznych lub ograniczeń przyrządu na wynik pomiaru.

Obie metody służą do obliczenia niepewności złożonej opisującej całkowity wpływ wszystkich źródeł na wynik pomiaru obliczana metodą propagacji niepewności. Dzięki której możemy obliczyć niepewność rozszerzoną $U = k(\text{rząd niepewności}) \cdot u_c$ reprezentując wynik $\pm U$

$$u_c(y) = \sqrt{\sum_{i=1}^n \left(\frac{\partial f}{\partial x_i} \cdot u(x_i) \right)^2}$$

22. Omówić sposób wyznaczenia parametrów sygnałów na podstawie ich reprezentacji dyskretnej.

Wyznaczanie parametrów sygnałów na podstawie **reprezentacji dyskretnej** polega na analizie próbkowanego sygnału cyfrowego w celu określenia jego charakterystycznych cech, takich jak amplituda, wartość średnia, moc, częstotliwość czy widmo.

Obliczamy:

1. Wartości średnie sygnału
2. Wartość skuteczną

$$x_{\text{rms}} = \sqrt{\frac{1}{N} \sum x[n]^2}$$

3. Amplituda maksymalna (maksymalna wartość bezwzględna z próbek sygnału)
4. Energię i moc – $E = \text{suma kwadratów próbek}$, $P = E / \text{ilość próbek}$
5. Częstotliwość i fazę

Wyznaczenie częstotliwości metodą zero-przebiegów:

Liczymy liczbę próbek między kolejnymi punktami, gdzie sygnał przechodzi przez zero w tym samym kierunku (tutaj od - do +):

{0,70,100,70,0,-70,-100,-70} w tym przykładzie 4 – 0,70,100,70,0

Liczymy okres – $T = \text{liczba próbek} \cdot \text{Okres próbkowania (czas między próbkami)}$

Częstotliwość = $1/T$

Wyznaczanie fazy metodą FFT:

Liczymy dyskretną transformatę fouriera dla próbek:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0..N-1$$

Następnie liczymy gdzie Im to część urojona liczby zespolonej a Re - rzeczywista:

$$\phi = \arctan \frac{\text{Im}(X[k])}{\text{Re}(X[k])} \approx \pi/6 (30^\circ)$$

23. Czym charakteryzują się klasy abstrakcyjne? Wyjaśnić na przykładzie języka C++

Klasy abstrakcyjne w C++ to klasy, które **nie mogą być instancjonowane** i służą do definiowania interfejsu dla klas pochodnych. Charakteryzują się tym, że zawierają **co najmniej jedną funkcję wirtualną czysto abstrakcyjną**.

- **Zawiera funkcje wirtualne czysto abstrakcyjne** (deklarowane z `= 0`), które muszą być **nadpisane w klasach pochodnych**, jeśli chcemy stworzyć obiekt pochodnej.
- **Może zawierać funkcje zwykłe** (z implementacją) i zmienne członkowskie.
- **Służy jako interfejs** dla grupy klas pochodnych – wymusza spójne zachowanie.

W C++ funkcje klasy oznaczane są słowem klucz `virtual` gdzie w javie funkcje są domyślnie wirtualne – ogranicza się je użyciem słowem klucz `final`.

W implementacji klasy pochodnej wykorzystujemy słowo klucz `virtual` do określenia zachowania funkcji abstrakcyjnej lub możemy wykonać implementację klasy rodzica.

```
1 class Figura {
2 public:
3     virtual double pole() = 0; // funkcja czysto wirtualna
4     virtual void info() { cout << "To jest figura." << endl; } // funkcja zwykła
5 };
6
7 // Klasa pochodna
8 class Prostokat : public Figura {
9 private:
10     double a, b;
11 public:
12     Prostokat(double x, double y) : a(x), b(y) {}
13     double pole() override { return a * b; } // implementacja funkcji abstrakcyjnej
14 };
```

24. Do czego służy mechanizm przeciążania funkcji i metod? Wyjaśnić na przykładzie języka C++

Mechanizm przeciążania funkcji i metod w C++ pozwala definiować **wiele funkcji o tej samej nazwie**, które różnią się **typem lub liczbą parametrów**. Służy to **czytelności kodu i elastyczności wywołań**, umożliwiając tę samą nazwę dla podobnych operacji.

Cechy przeciążania

1. Funkcje/metody mają **taką samą nazwę**.
2. Muszą się **różnić liczbą lub typem parametrów**.
3. Nie można przeciążać funkcji tylko **po typie zwracanym**.
4. Pozwala pisać bardziej **czytelny i intuicyjny kod** – ta sama operacja działa na różnych typach danych.

```
1 int suma(int a, int b) {return a + b;}
2 double suma(double a, double b) {return a + b;}
3 int suma(int a, int b, int c) {return a + b + c;}
```

Przeciążenie metod:

```
1 class Prostokat {
2 private:
3     double a, b;
4 public:
5     Prostokat(double x, double y) : a(x), b(y) {}
6     double pole() { return a * b; } // bez parametrów
7     double pole(double scale) { return a * b * scale; } // z parametrem};
8 int main() {
9     Prostokat p(3,4);
10    cout << p.pole() << endl; // 12
11    cout << p.pole(2) << endl; // 24}
```

25. Porównać główne cechy architektur procesorów RISC i CISC

Cecha	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Zestaw instrukcji	Mały, prosty zestaw instrukcji	Duży, złożony zestaw instrukcji
Czas wykonania instrukcji	Zazwyczaj jedna instrukcja = 1 cykl zegara	Instrukcje mogą wymagać wielu cykli zegara
Skomplikowanie dekodera	Prosty dekodér – łatwe pipeline'owanie	Złożony dekodér, trudniejsze pipeline'owanie
Format instrukcji	Stała długość	Zmienna długość
Wykorzystanie rejestrów	Duża liczba rejestrów – operacje głównie na rejestrach	Mniejsza liczba rejestrów – częściej operacje na pamięci
Wydajność	Wysoka w jednostkach pipeline, przewidywalne czasy wykonania	Wysoka dla pojedynczych, złożonych operacji; mniej przewidywalne czasy wykonania
Kod programu	Dłuższy (więcej prostych instrukcji)	Krótszy (jedna złożona instrukcja może wykonać wiele operacji)

RISC – prostota, szybkie i przewidywalne instrukcje, pipeline, operacje głównie na rejestrach.

CISC – złożone instrukcje, mniej rejestrów, każda instrukcja może wykonywać wiele operacji, ale wykonanie może trwać różną liczbę cykli

26. Omówić główne organizacje architektur wieloprocessorowych

Organizacja architektur wieloprocessorowych opisuje sposób łączenia i współdziałania wielu procesorów w systemie. Główne typy można podzielić według **metody dostępu do pamięci i sposobu komunikacji między procesorami**.

1. Klasyfikacja według dostępu do pamięci

a) SMP – Symmetric Multiprocessing (symetryczne)

- Wszystkie procesory mają **równy dostęp do wspólnej pamięci głównej**.

- Każdy procesor może wykonywać dowolne zadanie.
- Stosuje się **coherent cache** – spójność pamięci podręcznej.
- Przykład: większość nowoczesnych serwerów x86.

Cechy:

- Proste programowanie (globalna pamięć)
- Ograniczona skalowalność – przy dużej liczbie procesorów występuje konflikt o dostęp do pamięci

b) NUMA – Non-Uniform Memory Access

- Każdy procesor ma własną pamięć lokalną i dostęp do pamięci innych procesorów, ale **czas dostępu zależy od lokalizacji**.
- Zwiększa skalowalność w dużych systemach.
- Wymaga systemu operacyjnego świadomego rozmieszczenia danych.

c) MPP – Massively Parallel Processing

- Każdy procesor ma własną pamięć i komunikacja odbywa się przez **sieć połączeń**.
- Brak wspólnej pamięci – programowanie wymaga rozdzielenia danych.
- Przykłady: klastry superkomputerów, systemy GPU HPC.

2. Klasyfikacja według modelu komunikacji (Flynn)

Typ	Opis	Przykład
SISD	Single Instruction, Single Data – klasyczny, jeden procesor	zwykły komputer jednoukładowy
SIMD	Single Instruction, Multiple Data – jedna instrukcja operuje równocześnie na wielu danych	GPU, wektoryzacja
MISD	Multiple Instruction, Single Data – wiele instrukcji na tych samych danych	rzadko stosowane, np. w systemach redundancji krytycznej
MIMD	Multiple Instruction, Multiple Data – każdy procesor wykonuje własny program na własnych danych	SMP, NUMA, MPP

Organizacja	Dostęp do pamięci	Skalowalność	Trudność programowania
SMP	wspólna pamięć	umiarkowana	niska
NUMA	pamięć lokalna + zdalna	wysoka	średnia
MPP	pamięć prywatna, sieć	bardzo wysoka	wysoka

-
- **SMP** – łatwe w użyciu, ograniczone skalowanie.
 - **NUMA** – szybki dostęp lokalny, wymaga świadomego rozmieszczenia danych.
 - **MPP** – bardzo duża skalowalność, każdy procesor niezależny, wymaga rozdzielenia zadań i danych.

27. Omówić trójwarstwową architekturę ANSI-SPARC wykorzystywaną w relacyjnych baz danych.

Trójwarstwowa architektura ANSI-SPARC to klasyczny model organizacji **relacyjnych baz danych**, wprowadzony w latach 70. przez ANSI i SPARC. Jej celem jest **oddzielenie fizycznego przechowywania danych od sposobu ich prezentacji i użytkowania**, co zwiększa niezależność i elastyczność systemu.

1. Trzy poziomy architektury

a) Poziom zewnętrzny (External Level) – widoki użytkownika

- Reprezentuje **widoki danych dla różnych użytkowników lub aplikacji**.
- Każdy użytkownik może widzieć **tylko potrzebną część danych** (np. tylko swoje zamówienia w bazie firmowej).
- **Zaleta:** bezpieczeństwo i izolacja danych.

b) Poziom pośredni (Conceptual Level) – schemat pośredni

- Reprezentuje **całą strukturę logiczną bazy danych**.
- Określa:
 - encje i relacje między nimi,
 - ograniczenia integralności,
 - typy danych.
- Jest **niezależny od sposobu fizycznego przechowywania** danych.

c) Poziom wewnętrzny (Internal Level) – schemat fizyczny

- Odpowiada za **fizyczną organizację danych na nośniku**: pliki, indeksy, struktury bloków.
- Zawiera informacje o:
 - sposobie przechowywania rekordów,
 - organizacji plików,
 - metodach dostępu i optymalizacji.
- Jest **niewidoczny dla użytkownika**.

3. Zalety architektury ANSI-SPARC

1. **Niezależność danych logiczna** – zmiana schematu fizycznego nie wymaga zmiany aplikacji użytkownika.
2. **Niezależność danych fizyczna** – zmiana sposobu przechowywania (np. indeksów) nie wpływa na schemat pośredni.
3. **Bezpieczeństwo i kontrola dostępu** – różne widoki dla różnych użytkowników.
4. **Elastyczność i skalowalność** – łatwiejsze utrzymanie dużych systemów bazodanowych.

28. Omówić diagram związków encji (ang. Entity-Relationship Diagram).

Diagram związków encji (ERD – Entity-Relationship Diagram) to graficzna reprezentacja danych i ich powiązań w systemie. Stosuje się go głównie przy projektowaniu **relacyjnych baz danych**.

1. Podstawowe elementy ERD

Element	Opis	Przykład
Encja (Entity)	Obiekt lub pojęcie, o którym przechowujemy dane	Pracownik, Samochód
Atrybut (Attribute)	Właściwość encji	imię, nazwisko, PESEL
Klucz główny (Primary Key)	Atrybut identyfikujący jednoznacznie encję	ID_Pracownika
Związek (Relationship)	Powiązanie między encjami	Pracuje w, Wypożycza
Krotność (Cardinality)	Określa ile instancji jednej encji może być powiązanych z drugą	1:1, 1:N, N:M

2. Typy związków

- **1:1** – każda instancja encji A jest powiązana z maksymalnie jedną instancją encji B
- **1:N** – jedna instancja A może być powiązana z wieloma instancjami B
- **N:M** – wiele instancji A może być powiązanych z wieloma instancjami B (wymaga tabeli pośredniej w relacyjnej bazie)

3. Zalety diagramu ER

1. Ułatwia **projektowanie bazy danych** i jej struktury.
2. Pokazuje **powiązania między danymi** w czytelny sposób.
3. Pomaga w określeniu **kluczy głównych i obcych**.
4. Stanowi **dokumentację dla programistów i analityków**.

W ERD można rysować prostokąty dla encji, elipsy dla atrybutów, romby dla związków i oznaczać krotności.

29. Omówić podstawowe elementy architektury procesora, ich znaczenie i rolę w przetwarzaniu danych.

1. Jednostka arytmetyczno-logiczna (ALU – Arithmetic Logic Unit)

- Wykonuje operacje **arytmetyczne** (dodawanie, odejmowanie, mnożenie, dzielenie) i **logiczne** (AND, OR, NOT, XOR).
- **Znaczenie:** serce przetwarzania danych – wszystkie obliczenia i decyzje logiczne są realizowane tutaj.

2. Jednostka sterująca (CU – Control Unit)

- Odpowiada za **koordynację pracy procesora**: pobiera instrukcje z pamięci, dekoduje je i wysyła sygnały sterujące do ALU i rejestrów.
- **Znaczenie:** zapewnia prawidłową kolejność wykonywania instrukcji i synchronizację komponentów procesora.

3. Rejestry

- Małe, bardzo szybkie pamięci wewnętrzne procesora do przechowywania danych tymczasowych i adresów.
- Typy rejestrów:
 - **Rejestry ogólnego przeznaczenia** – przechowują dane i wyniki obliczeń.
 - **Rejestry specjalne** – np. licznik rozkazów (PC), rejestr instrukcji (IR), rejestr statusu (FLAGS).
- **Znaczenie:** szybki dostęp do danych i adresów potrzebnych w trakcie wykonywania instrukcji.

4. Pamięć podręczna (Cache)

- Mała, szybka pamięć między procesorem a RAM-em.
- Przechowuje często używane dane i instrukcje.
- **Znaczenie:** redukuje czas oczekiwania na dane z pamięci głównej, zwiększa wydajność przetwarzania.

5. Szyna danych i adresowa

- **Szyna danych** – przenosi dane między komponentami procesora i pamięcią.
- **Szyna adresowa** – określa lokalizację pamięci, z której dane mają być odczytane lub zapisane.
- **Znaczenie:** umożliwia przepływ danych i komunikację z pamięcią i urządzeniami wejścia/wyjścia.

6. Dekoder instrukcji

- Odczytuje instrukcję z rejestru instrukcji (IR) i generuje sygnały sterujące dla ALU, rejestrów i szyn.
- **Znaczenie:** pozwala procesorowi “rozumieć” i wykonywać instrukcje programu.

7. Jednostka wejścia/wyjścia (I/O)

- Umożliwia komunikację procesora z urządzeniami zewnętrznymi (klawiatura, monitor, dysk).
- **Znaczenie:** wymiana danych między procesorem a światem zewnętrznym.

Podsumowanie:

Procesor przetwarza dane dzięki **ALU** (operacje), **CU** (koordynacja), **rejestrom** i **cache** (szybki dostęp do danych), a szyny i jednostki I/O umożliwiają komunikację z pamięcią i urządzeniami zewnętrznymi. Każdy element pełni konkretną rolę w cyklu pobierania, dekodowania i wykonywania instrukcji.

30. Przedstawić cykl rozkazowy procesora, jego fazy i realizowane w nich mikrooperacje

Cykl rozkazowy procesora

1. Fazy cyklu

1. Pobranie (Fetch)

- Pobranie instrukcji z pamięci do rejestru instrukcji (IR)
- Zwiększenie licznika rozkazów (PC)
- Mikrooperacje: odczyt z pamięci, zapis do IR, aktualizacja PC

2. Dekodowanie (Decode)

- Analiza instrukcji (opcode)
- Określenie adresów operandów
- Przygotowanie sygnałów sterujących

3. Wykonanie (Execute)

- Realizacja operacji w ALU
- Odczyt/zapis danych z/do rejestrów
- Aktualizacja rejestrów statusu

4. Dostęp do pamięci (Memory Access, jeśli potrzebny)

- Pobranie lub zapis danych w pamięci

5. Zapis wyniku (Write Back, jeśli potrzebny)

- Zapis wyniku do rejestru docelowego
- Aktualizacja PC przy skokach

2. Schemat

[Fetch] → [Decode] → [Execute] → [Memory Access] → [Write Back] →
(następna instrukcja)

- **Fazy Fetch i Decode** występują przy każdej instrukcji.
- **Memory Access i Write Back** tylko dla instrukcji operujących na pamięci lub zapisujących wynik.

31. Omówić koncepcje programowania zdarzeniowego graficznych interfejsów użytkownika

Program czeka na zdarzenia (kliknięcia , naciśnięcia, zmiany wartości)

Każde zdarzenie (event) wywołuje odpowiednią funkcję (handlera). Obiekt generujący zdarzenie nazywamy Event Source – okno , przycisk , pole tekstowe.

Całość wykonywana jest w pętli zdarzeń (Event Loop) – oczekuje ona na zdarzenia i przekazuje je do handlerów.

Pozwala na reaktywny, modularny/ elastyczny i pozwala na rozdzielenie logiki każdego obiektu GUI.

32. Przedstawić sposób obsługi myszy i klawiatury w aplikacji stworzonej w oparciu o model programowania zdarzeniowego

W aplikacji (event loopie) Podczas kliknięcia myszy (event) na event source (przycisk , label lub cokolwiek) wywołujemy funkcję (handlera)

33. Omówić zalety przedstawiania układu ciągłego w przestrzeni stanów

Przedstawienie układu ciągłego w **przestrzeni stanów** polega na opisaniu dynamiki układu za pomocą wektora stanu ($x(t)$), który zawiera wszystkie zmienne potrzebne do pełnego określenia stanu układu w danym momencie. Równania mają postać:

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t)$$

gdzie ($x(t)$) – wektor stanu, ($u(t)$) – wejście, ($y(t)$) – wyjście, a (A , B , C , D) – macierze opisujące układ.

Zastosowania i zalety w praktyce:

1. **Analiza stabilności** – można łatwo sprawdzić, czy układ będzie stabilny, badając wartości własne macierzy (A).
2. **Sterowalność i obserwowalność** – pozwala określić, czy układ da się sterować i czy można odtworzyć jego stan na podstawie pomiarów.
3. **Projektowanie regulatorów** – np. LQR (optymalny regulator liniowy) lub obserwatorów stanu, które trudno uzyskać z klasycznych równań różniczkowych wejście–wyjście.
4. **Łączenie układów** – przy opisie w przestrzeni stanów łatwo łączyć bloki w jeden większy układ.
5. **Symulacja komputerowa** – równania w przestrzeni stanów łatwo implementować w MATLAB/Simulink lub w programowaniu numerycznym.

34. Omówić zasady stosowania programowania dynamicznego

Zasady stosowania **programowania dynamicznego (PD)**:

1. **Rozbicie problemu na podproblemy** – duży problem dzieli się na mniejsze, które można rozwiązać niezależnie.
2. **Optymalność podstruktur (zasada Bellmana)** – optymalne rozwiązanie całego problemu można uzyskać z optymalnych rozwiązań jego podproblemów.
3. **Zapamiętywanie wyników (memoizacja)** – wyniki podproblemów są przechowywane, aby uniknąć wielokrotnego przeliczania.
4. **Iteracyjne lub rekurencyjne rozwiązanie** – problem rozwiązuje się od najprostszych podproblemów w górę (bottom-up) lub rekurencyjnie (top-down).
5. **Zastosowanie w różnych dziedzinach** – PD stosuje się w planowaniu, sterowaniu optymalnym, problemach kombinatorycznych, np. plecak, najkrótsza ścieżka, edycja ciągów.

Fibonacci – Tworzymy tablicę , ustawiamy $DP[0] = 0$, $dp[1] = 1$ iteracyjnie liczymy dla $i = 2..n$

$$dp[i] = dp[i-1] + dp[i-2]$$

35. Omówić tryby adresowania procesora w architekturach typu RISC i CISC

1. RISC (Reduced Instruction Set Computer):

- Charakteryzuje się **prostymi i jednolitymi instrukcjami**.
- Główne tryby adresowania:
 - **Rejestrowy (Register addressing)**: operand w rejestrze procesora.
 - **Bezpośredni (Immediate addressing)**: wartość podana w instrukcji.
 - **Pośredni (Register indirect)**: adres w rejestrze wskazuje pamięć.
 - **Adresowanie bazowe z przesunięciem (Base + Offset)**: np. $R1 + offset$ wskazuje lokalizację w pamięci.
- Cel: minimalizowanie złożoności dekodowania i przyspieszenie wykonania.

2. CISC (Complex Instruction Set Computer):

- Ma **rozbudowany zestaw instrukcji**, często obsługujący złożone operacje jednym rozkazem.
- Typowe tryby adresowania:
 - **Rejestrowy**
 - **Natychmiastowy**
 - **Bezpośredni (Direct)**: adres w pamięci podany w instrukcji.
 - **Pośredni (Indirect)**: adres przechowywany w rejestrze lub pamięci.
 - **Indeksowy (Indexed)**: adres bazowy + rejestr indeksowy.
 - **Relatywny (Relative)**: stosowany np. w skokach.
- Pozwala na większą elastyczność i skrócenie kodu, kosztem dłuższego dekodowania.

Różnica kluczowa:

- RISC – prostota, jednolitość i szybkie wykonanie, ograniczona liczba trybów.
- CISC – duża liczba trybów, bardziej złożone instrukcje, oszczędność pamięci kodu kosztem szybkości dekodowania.

36. Przedstawić mikrokontrolery i ich układy peryferyjne na przykładzie wybranej architektury **Arduino Uno** (mikrokontroler oparty na **ATmega328P**).

1. Rdzeń CPU:

- Mikrokontroler **8-bitowy AVR**.
- Zegar: 16 MHz.
- Architektura RISC – proste instrukcje, szybkie wykonanie.

2. Pamięć:

- Flash: 32 KB (program), z czego 0,5 KB zajmuje bootloader.
- SRAM: 2 KB (dane).
- EEPROM: 1 KB (dane trwałe).

3. Układy peryferyjne wbudowane w ATmega328P:

- **GPIO** – cyfrowe wejścia/wyjścia (14 pinów).
- **PWM** – generowanie sygnałów sterujących silnikami lub diodami LED (6 pinów).
- **ADC** – przetwornik analogowo-cyfrowy 10-bitowy (6 wejść).
- **UART, SPI, I2C** – komunikacja szeregową.
- **Timery / Watchdog** – odmierzanie czasu, nadzór nad programem.

4. Schemat działania:

CPU steruje peryferiami poprzez rejestry wbudowane w mikrokontroler. Program może np.:

- Odczytać czujnik temperatury przez ADC,
- Włączyć diodę LED przez GPIO,
- Sterować serwomechanizmem przez PWM,
- Wysłać dane do komputera przez UART/USB.

37. Porównać kompozycję oraz dziedziczenie w języku JAVA

Cecha	Dziedziczenie	Kompozycja
Definicja	Klasa potomna przejmuję właściwości i metody klasy bazowej (<code>extends</code>).	Klasa posiada obiekty innych klas jako swoje pola (zależność „ma”).
Relacja	„Jest rodzajem” (<code>is-a</code>).	„Ma w sobie” (<code>has-a</code>).
Ponowne użycie kodu	Przez rozszerzenie istniejącej klasy.	Przez delegowanie zadań do składowych obiektów.
Elastyczność	Mniej elastyczne – ograniczenie jednej klasy bazowej (Java nie wspiera wielodziedziczenia klas).	Bardziej elastyczne – można komponować wiele obiektów, zmieniać w runtime.
Sprzyja polimorfizmowi	Tak, metody klasy bazowej mogą być nadpisywane.	Tak, jeśli używamy interfejsów i delegacji.
Ryzyko problemów	Dziedziczenie głębokie może prowadzić do silnego sprzężenia i trudnego utrzymania kodu.	Mniejsze ryzyko – luźne powiązanie między klasami.

Dziedziczenie – dobre, gdy mamy **naturalną hierarchię typu „is-a”** i chcemy używać polimorfizmu.

Kompozycja – lepsza, gdy chcemy **elastyczności i luźnego powiązania** („favor composition over inheritance”). – polimorfizm za pomocą dependency injection

38. Omówić podstawowe kolekcje obiektów występujące w języku JAVA: listy , zbiory oraz mapy

1. Listy (**List**)

- Przechowują elementy **w określonej kolejności**, mogą zawierać **duplikaty**.
- Najczęściej używane implementacje:
 - `ArrayList` – szybki dostęp po indeksie, wolniejsze wstawianie/usuwanie w środku listy.
 - `LinkedList` – szybkie wstawianie/usuwanie w środku, wolniejszy dostęp po indeksie.

2. Zbiory (**Set**)

- Nie przechowują elementów w określonej kolejności (chyba że używamy `LinkedHashSet` lub `TreeSet`).
- **Nie dopuszczają duplikatów.**
- Najczęściej używane implementacje:
 - `HashSet` – szybkie dodawanie i wyszukiwanie, brak kolejności.
 - `LinkedHashSet` – zachowuje kolejność dodawania.
 - `TreeSet` – uporządkowany zbiór, elementy posortowane.

3. Mapy (**Map**)

- Przechowują pary **klucz → wartość**.

- Każdy klucz może występować **tylko raz**, wartości mogą się powtarzać.
- Najczęściej używane implementacje:
 - `HashMap` – szybkie wyszukiwanie po kluczu, brak gwarancji kolejności.
 - `LinkedHashMap` – zachowuje kolejność dodawania.
 - `TreeMap` – uporządkowane według klucza.

39. Scharakteryzować strukturę działania i algorytm uczenia pojedynczego sztucznego neuronu ze skokową funkcją aktywacji

Okej, omówmy **pojedynczy sztuczny neuron (perceptron) ze skokową funkcją aktywacji** w sposób zwięzły i przejrzysty.

1. Struktura neuronu

- **Wejścia:** $(x_1, x_2, \dots, x_n)$ – wartości sygnałów wejściowych.
- **Wagi:** $(w_1, w_2, \dots, w_n)$ – współczynniki wpływające na znaczenie każdego wejścia.
- **Suma ważona:**

$$z = \sum_{i=1}^n w_i x_i + b$$
 gdzie (b) – bias (próg).
- **Funkcja aktywacji:** skokowa (Heaviside):

$$y = \begin{cases} 1 & \text{jeśli } z \geq 0 \\ 0 & \text{jeśli } z < 0 \end{cases}$$
- **Wyjście:** (y) – wynik neuronu (0 lub 1).

2. Działanie neuronu

1. Każde wejście jest mnożone przez swoją wagę.
 2. Sumuje się wszystkie ważone wejścia i dodaje bias.
 3. Przez funkcję skokową generowane jest wyjście 0 lub 1.
 4. Neuron działa jako **klasyfikator liniowy** – rozdziela przestrzeń wejść prostą graniczą.
-

3. Algorytm uczenia perceptronu

Celem jest dopasowanie wag i biasu tak, aby neuron poprawnie klasyfikował dane.

Kroki algorytmu:

1. **Inicjalizacja** wag (w_i) i biasu (b) (np. losowo lub zerowo).
2. **Dla każdego przykładu uczącego:**

- Oblicz wyjście: $y = f(\sum w_i x_i + b)$
- Oblicz błąd: $e = d - y$ (gdzie d – wartość oczekiwana)
- Zaktualizuj wagi i bias:
$$w_i := w_i + \eta \cdot e \cdot x_i$$
$$b := b + \eta \cdot e$$

gdzie η – współczynnik uczenia.

3. Powtarzaj kroki 2–3 do momentu, aż wszystkie przykłady zostaną poprawnie sklasyfikowane lub osiągnięty zostanie maksymalny liczba iteracji.

4. Właściwości

- Działa tylko dla **danych liniowo separowalnych**.
- Jest podstawą sieci wielowarstwowych (MLP), gdzie stosuje się bardziej złożone funkcje aktywacji.
- Skokowa funkcja aktywacji powoduje, że gradient nie istnieje – klasyczny perceptron **uczy się iteracyjnie, a nie metodą gradientu**.

40. Scharakteryzować rolę warstwy ukrytej (warstw ukrytych) w funkcjonowaniu sztucznej sieci neuronowej typu perceptron wielowarstwowy.

W **perceptronie wielowarstwowym (MLP)** warstwy ukryte odgrywają kluczową rolę w przetwarzaniu informacji. Oto charakterystyka:

1. Rola warstw ukrytych

- **Ekstrakcja cech** – warstwy ukryte przekształcają sygnały wejściowe w bardziej złożone reprezentacje (cechy).
- **Rozwiązywanie problemów nieliniowych** – pojedynczy perceptron działa tylko dla danych liniowo separowalnych; warstwy ukryte pozwalają sieci aproksymować **funkcje nieliniowe**.
- **Przekazywanie informacji** – każda warstwa ukryta odbiera wyjścia poprzedniej warstwy, przetwarza je i przekazuje dalej.
- **Umożliwienie głębokiego uczenia** – wiele warstw ukrytych pozwala sieci uczyć hierarchii cech, od prostych do coraz bardziej złożonych.

2. Struktura warstwy ukrytej

- Składa się z neuronów, każdy z własnymi wagami i biasem.

- Każdy neuron w warstwie ukrytej stosuje **funkcję aktywacji nieliniową** (sigmoid, ReLU itp.).
- Wyjścia neuronów warstwy ukrytej stają się wejściami dla kolejnej warstwy lub warstwy wyjściowej.

3. Znaczenie praktyczne

- Bez warstw ukrytych MLP redukuje się do pojedynczego perceptronu → nie poradzi sobie z problemami nieliniowymi, np. XOR.
- Liczba neuronów i warstw ukrytych wpływa na **zdolność sieci do aproksymacji funkcji i jej moc generalizacji**.

Podsumowanie:

Warstwy ukryte w MLP służą do **tworzenia nieliniowych kombinacji cech wejściowych**, umożliwiając sieci uczenie się **złożonych zależności** między wejściami a wyjściami.

41. Omówić budowę i działanie całkowicie sprawiedliwego planisty procesów – CFS

Używany w jądrze linuxa

1. Cel CFS

- Zapewnia **sprawiedliwe przydzielanie czasu CPU** wszystkim procesom w systemie.
- Każdy proces otrzymuje proporcjonalną ilość czasu procesora względem swojej wagi/prioritetu.
- Minimalizuje opóźnienia i zagłócenie procesów.

2. Budowa

- **Run queue:** CFS przechowuje wszystkie procesy gotowe do uruchomienia w **drzewie czerwono-czarnym (RB-tree)**, posortowanym według **vruntime** (virtual runtime).
- **vruntime:** licznik czasu CPU „przepracowanego” przez proces, przeliczany z uwzględnieniem priorytetu:

$$vruntime = \frac{rzeczywisty_czas_CPU}{waga_procesu}$$
- Proces z najmniejszym vruntime jest **następnym wybranym procesem do wykonania**.

3. Działanie CFS

1. Każdy proces startuje z **vruntime = 0**.
2. Gdy proces korzysta z CPU, jego **vruntime** rośnie proporcjonalnie do czasu CPU i wagi.
3. Po zakończeniu kwantu czasu CFS wybiera **proces z najmniejszym vruntime** z drzewa RB.
4. Drzewo RB pozwala na szybkie wyszukiwanie, wstawianie i usuwanie procesów ($O(\log n)$).
5. Procesy w CFS są **dzielone proporcjonalnie do priorytetu**, dzięki czemu każdy proces dostaje „sprawiedliwy” udział CPU.

4. Zalety CFS

- **Sprawiedliwość:** każdy proces otrzymuje proporcjonalny czas CPU.
- **Skalowalność:** dzięki drzewu RB wybór procesu trwa logarytmicznie w stosunku do liczby procesów.
- **Brak zagłódzenia:** procesy o niższym priorytecie nadal dostają czas CPU.
- **Elastyczność:** można dynamicznie zmieniać wagi/prioritety procesów.

5. Podsumowanie

CFS to scheduler oparty na **sprawiedliwości wirtualnego czasu** (vruntime), wykorzystujący **drzewo czerwono-czarne** do efektywnego wyboru procesu. Proces z najmniejszym vruntime jest zawsze wybrany, co zapewnia proporcjonalny i równomierny dostęp do CPU.

42. Omówić budowę i działanie terminowego planisty żądań I/O

1. Cel planisty terminowego (Deadline)

- Zapewnia **terminowe wykonywanie operacji I/O** (odczytu/zapisu dysku).
- Minimalizuje **opóźnienia oczekiwania** na dysk i **zapobiega zagłódzeniu procesów**.
- Stosowany głównie w systemach, gdzie istotny jest czas odpowiedzi na żądania I/O.

2. Budowa planisty

Planista Deadline utrzymuje **dwa główne typy kolejek żądań I/O**:

1. **Read queue (kolejka odczytu)** – żądania odczytu, które mają najwyższy priorytet.
2. **Write queue (kolejka zapisu)** – żądania zapisu, które mają nieco niższy priorytet.

Każde żądanie I/O posiada:

- **Logical block address (LBA)** – adres bloku na dysku.
- **Deadline (termin)** – maksymalny czas oczekiwania, po którym żądanie musi być obsłużone.
- **Czas zgłoszenia** – do obliczania opóźnienia.

3. Działanie algorytmu

1. Wszystkie żądania I/O trafiają do odpowiedniej kolejki (odczyt/zapis).
2. Planista sortuje żądania według **adresu logicznego** (SCAN/LOOK – minimalizuje ruch głowicy) i **deadline'u**.
3. Podczas wyboru żądania do obsługi:
 - Jeśli żądanie osiągnęło swój **deadline**, jest obsługiwane natychmiast – **zapobiega zagłódzeniu**.
 - W przeciwnym razie planista stara się wybrać żądanie minimalizujące ruch głowicy (optymalizacja przepustowości).
4. Proces powtarza się dynamicznie, gdy przychodzą nowe żądania.

4. Zalety planisty Deadline

- **Krótkie czasy oczekiwania** dla krytycznych żądań (terminowych).
- **Zapobiega zagłódzeniu** procesów o niskim priorytecie.
- **Dobry kompromis** między sprawiedliwością a przepustowością dysku.

5. Podsumowanie

Planista terminowy I/O zapewnia, że:

- żądania krytyczne są realizowane na czas,
- inne żądania są sortowane tak, aby minimalizować ruch głowicy dysku,
- cały system I/O działa **sprawiedliwie i wydajnie**.

43. Omówić 3 problemy programowania współbieżnego – zakleszczenie, zagłodzenie i livelock

1. Zakleszczenie (Deadlock)

- **Definicja:** sytuacja, w której **dwa lub więcej procesów czeka na zasoby zajęte przez siebie nawzajem**, i żaden z nich nie może kontynuować.
- **Przykład:** proces A trzyma mutex1 i czeka na mutex2, proces B trzyma mutex2 i czeka na mutex1.

2. Zagłodzenie (Starvation)

- **Definicja:** proces **nie dostaje dostępu do potrzebnych zasobów przez długi czas**, mimo że system wciąż działa.
- **Przyczyna:** inne procesy mają wyższy priorytet lub często zabierają zasoby, uniemożliwiając postęp procesowi niskiego priorytetu.
- **Przykład:** kolejka z priorytetami – procesy niskiego priorytetu mogą nigdy nie zostać wykonane, jeśli stale przychodzą procesy wyższego priorytetu.

3. Livelock

- **Definicja:** procesy **ciągle zmieniają stan w reakcji na inne procesy**, ale **żaden nie robi faktycznego postępu**.
- **Różnica od deadlock:** w deadlocku procesy są w stanie oczekiwania, w livelock procesy aktywnie „pracują”, ale bez efektu.
- **Przykład:** dwa procesy próbują przejść przez wąski korytarz, ciągle ustępują sobie nawzajem, ale żaden nie przechodzi.

Problem	Czy procesy aktywnie wykonują kod?	Przyczyna główna
Deadlock	Nie (czekają w nieskończoność)	Wzajemne blokowanie zasobów
Starvation	Tak, ale nie robią postępu	Niesprawiedliwy harmonogram / priorytety
Livelock	Tak, ciągle się zmieniają	Wzajemne dostosowywanie się procesów bez postępu

44. Omówić algorytm Dekkera w programowaniu współbieżnym

algorytm Dekkera, rozwiązanie problemu **współbieżnego dostępu do sekcji krytycznej** dla dwóch procesów.

1. Cel algorytmu Dekkera

- Zapewnienie **bezpiecznego dostępu do sekcji krytycznej** przez dwa procesy (P0 i P1) bez użycia mechanizmów sprzętowych (blokad, semaforów).
- Spełnia trzy podstawowe warunki:

1. **Wzajemne wykluczanie (mutual exclusion)** – w sekcji krytycznej może być tylko jeden proces.
2. **Brak zagłodzenia (no starvation)** – każdy proces w końcu wejdzie do sekcji krytycznej.
3. **Postęp (progress)** – jeśli żaden proces nie jest w sekcji krytycznej, decyzja o wejściu zapada w skończonym czasie.

2. Struktura algorytmu

Proces chce wejść → ustawia swoją flagę.

- Jeśli drugi proces też chce wejść → sprawdzana jest zmienna `turn`.
- Jeśli kolej należy do drugiego → ustępuje i czeka.
- Jeśli kolej należy do niego → wchodzi do sekcji krytycznej.
- Po wyjściu z sekcji krytycznej → przekazuje kolej i resetuje flagę.
- Następnie wykonuje sekcję niekrytyczną i pętla zaczyna się od nowa.

Cecha	Algorytm Dekkera	Typowy mutex
Implementacja	Programowa, nie wymaga sprzętu	Może być sprzętowy lub systemowy (np. <code>pthread_mutex</code>)
Liczba procesów	Opracowany dla dwóch procesów	Może obsługiwać wiele wątków/procesów
Złożoność	Trzeba ręcznie ustawiać flagi i kolej	Wystarczy wywołać <code>lock()</code> i <code>unlock()</code>
Zagrożenia	Wrażliwy na błędy implementacji, może spowolnić system	Sterowany przez jądro/system – bezpieczniejszy
Czekanie	Busy-wait (proces w pętli czeka)	Może być blokujące (proces śpi) lub nieblokujące

45. Omówić sposób pobrania zasobu z serwera protokołu http

Pobranie zasobu z serwera HTTP polega na wykonaniu transakcji **żądanie–odpowiedź** między klientem a serwerem.

1. Identyfikacja zasobu

- Zasób jest wskazywany przez **URL**, np.
`http://example.com/index.html`
- URL zawiera: protokół, adres serwera, opcjonalnie port i ścieżkę do zasobu.

2. Nawiązanie połączenia

- Klient (np. przeglądarka) nawiązuje połączenie **TCP** z serwerem (domyślnie port 80).
- W HTTP/1.1 połączenie może być **utrzymywane** (keep-alive).

3. Wysłanie żądania HTTP

Klient wysyła **żądanie HTTP**, najczęściej metodą **GET**:

GET /index.html HTTP/1.1

Host: example.com

Żądanie składa się z:

- linii startowej (metoda, ścieżka, wersja HTTP),
- nagłówków (np. Host, User-Agent),
- opcjonalnie ciała (dla GET zwykle brak).

4. Przetwarzanie po stronie serwera

- Serwer analizuje żądanie.
- Sprawdza uprawnienia, lokalizuje zasób lub generuje go dynamicznie.
- Przygotowuje odpowiedź.

5. Odpowiedź serwera

Serwer odsyła **odpowiedź HTTP**, np.:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 1234

<treść zasobu>

Odpowiedź zawiera:

- **kod statusu** (np. 200 OK, 404 Not Found),
- nagłówki,
- ciało odpowiedzi (treść zasobu).

6. Odbiór zasobu

- Klient odbiera odpowiedź.
- Zasób jest wyświetlany, zapisywany lub dalej przetwarzany (np. HTML, JSON, plik).

Podsumowanie

1. Klient zna URL zasobu
2. Nawiązuje połączenie z serwerem
3. Wysyła żądanie HTTP (GET)
4. Serwer przetwarza żądanie
5. Odsyła odpowiedź z kodem i treścią
6. Klient odbiera zasób

46. Omówić trzy przykładowy sposoby ataków na strony internetowe. Przedstawić sposoby zabezpieczenia przed tymi atakami. **trzy typowe ataki na strony internetowe** oraz **sposoby zabezpieczenia** przed nimi.

- **SQL Injection** – wstrzyknięcie kodu SQL przez dane wejściowe w celu uzyskania nieautoryzowanego dostępu do bazy danych;
ochrona: zapytania parametryzowane, walidacja danych.
- **XSS** – wstrzyknięcie złośliwego JavaScriptu wykonywanego w przeglądarce użytkownika;
ochrona: kodowanie danych wyjściowych, CSP.
- **DoS** – przeciążenie serwera dużą liczbą żądań w celu unieruchomienia strony;
ochrona: rate limiting, firewall, CDN.

DDOS – wariancja dosu z wielu źródeł jednocześnie – CDN i load balancing.

47. Wyjaśnić rolę oraz znaczenie sieci Virtual LAN

1. Definicja

- **VLAN to wirtualna sieć lokalna**, która dzieli fizyczną sieć LAN na **odizolowane segmenty logiczne**, niezależnie od fizycznej topologii.
- Każdy VLAN działa jak **oddzielna sieć**, nawet jeśli urządzenia są podłączone do tego samego switcha.

2. Rola VLAN

- **Izolacja ruchu sieciowego:** ruch w jednym VLAN-ie nie jest widoczny w innych VLAN-ach bez routera lub warstwy 3.
- **Poprawa bezpieczeństwa:** oddzielanie działów firmy (np. księgowość, IT) ogranicza ryzyko podsłuchu i ataków.
- **Redukcja broadcastów:** broadcasty są ograniczone do VLAN-u, co zmniejsza obciążenie sieci.
- **Elastyczność administracyjna:** łatwo przenosić urządzenia między VLAN-ami bez zmiany fizycznego okablowania.

3. Znaczenie praktyczne

- **Segmentacja sieci:** dział IT, księgowość i goście mogą korzystać z jednej infrastruktury fizycznej, ale mieć oddzielne VLAN-y.
- **QoS i priorytety ruchu:** ruch krytyczny może być przypisany do osobnego VLAN-u.
- **Zarządzanie dużą siecią:** w dużych firmach ułatwia kontrolę, monitorowanie i wdrażanie polityk bezpieczeństwa.

4. Podsumowanie

- VLAN to **logiczne wydzielenie sieci** w ramach jednej infrastruktury fizycznej.
- Umożliwia **bezpieczną izolację, lepszą wydajność i łatwiejsze zarządzanie**.
- W praktyce stosowany w firmach, centrach danych i sieciach korporacyjnych.

48. Omówić zagadnienia routingu statycznego oraz dynamicznego

1. Routing statyczny

- **Definicja:** administrator ręcznie konfiguruje trasy w tablicy routingu.
- **Zalety:**
 - prostota i przewidywalność,
 - brak dodatkowego obciążenia procesora i pasma sieci.
- **Wady:**
 - brak automatycznego dostosowania do zmian w sieci,
 - trudny do zarządzania w dużych sieciach.
- **Zastosowanie:** małe sieci, sieci z niewielką liczbą zmian topologii.

2. Routing dynamiczny

- **Definicja:** routery automatycznie wymieniają informacje o trasach i dostosowują tablice routingu w czasie rzeczywistym.
- **Zalety:**
 - automatyczna adaptacja do awarii i zmian w sieci,
 - skalowalność dla dużych i złożonych sieci.
- **Wady:**
 - większe zużycie CPU i pasma,
 - ryzyko błędów lub pętli routingu przy źle skonfigurowanych protokołach.
- **Protokoły:** RIP, OSPF, EIGRP, BGP.

3. Podsumowanie różnic

Cecha	Routing statyczny	Routing dynamiczny
Konfiguracja	Ręczna	Automatyczna
Adaptacja do zmian	Nie	Tak
Skalowalność	Mała	Duża
Obciążenie sieci	Niskie	Wyższe

49. Przedstawić wady i zalety kompilowanych i interpretowanych języków programowania. Wyjaśnić na czym polega dualna natura języka Python w tym zakresie

1. Języki kompilowane

- **Przykłady:** C, C++, Go.
- **Działanie:** kod źródłowy jest **kompilowany** do kodu maszynowego przed uruchomieniem.

Zalety:

- szybkie wykonywanie programu (kod maszynowy),
- optymalizacja przez kompilator,
- brak zależności od interpretera w czasie uruchomienia.

Wady:

- dłuższy czas tworzenia programu (kompilacja),
- trudniejsze debugowanie,
- mniejsza przenośność między platformami (wymaga rekompilacji).

2. Języki interpretowane

- **Przykłady:** JavaScript, Ruby, PHP.
- **Działanie:** kod źródłowy jest **wykonywany bezpośrednio przez interpreter** w czasie uruchomienia.

Zalety:

- łatwe testowanie i debugowanie,
- większa przenośność (działa tam, gdzie jest interpreter),
- szybki start programu (brak kompilacji).

Wady:

- wolniejsze wykonanie,
- większe zużycie pamięci,
- zależność od interpretera.

3. Dualna natura Pythona

- Python jest **językiem interpretowanym**, ale w praktyce działa w **dwóch etapach**:
 1. Kod źródłowy jest kompilowany do **bytecode'u (pośredni kod)**,
 2. Bytecode jest **wykonywany przez interpreter Pythona (CPython, PyPy itp.)**.

Cecha	Język kompilowany	Język interpretowany	Python
Wykonanie	szybkie	wolniejsze	pośrednie (bytecode + interpreter)
Debugowanie	trudniejsze	łatwiejsze	łatwe
Przenośność	wymaga rekompilacji	wysoka	wysoka dzięki bytecode
Uruchamianie	wymaga kompilacji	natychmiast	szybkie dzięki bytecode

50. Wyjaśnić znaczenie pojęcia edge computing

Edge computing to przetwarzanie danych **blisko źródła ich powstawania** (urządzeń IoT, sensorów, kamer) zamiast przesyłania wszystkiego do centralnego serwera lub chmury.

2. Znaczenie i zalety

- **Niższe opóźnienia (low latency):** odpowiedzi systemu są szybsze, bo dane nie muszą podróżować do chmury.
- **Oszczędność pasma sieci:** tylko istotne dane są przesyłane dalej.
- **Skalowalność:** przetwarzanie rozproszone zmniejsza obciążenie centralnych serwerów.
- **Bezpieczeństwo i prywatność:** wrażliwe dane mogą być analizowane lokalnie, zanim zostaną wysłane.

3. Przykłady zastosowań

- Inteligentne miasta – analiza ruchu i sygnalizacji świetlnej lokalnie.
- Samojedzne pojazdy – przetwarzanie danych z sensorów w czasie rzeczywistym.
- Przemysł 4.0 – szybka analiza danych z maszyn produkcyjnych.

Edge computing = **przetwarzanie danych „na krawędzi sieci”**, blisko źródła, co zmniejsza opóźnienia, oszczędza pasmo i poprawia bezpieczeństwo

51. Scharakteryzować wybrane platformy mikroprogramowalne stosowane w budowie systemów IoT

1. Arduino

- **Budowa:** mikrokontrolery z rodziny AVR (np. ATmega328P), proste wejścia/wyjścia cyfrowe i analogowe.
- **Zastosowanie w IoT:** sterowanie czujnikami, silnikami, prostymi urządzeniami, prototypowanie.
- **Zalety:**
 - prostota programowania w C/C++ (Arduino IDE),
 - duża społeczność i biblioteki,
 - niska cena.
- **Wady:** ograniczone zasoby (pamięć, moc obliczeniowa), brak wbudowanego Wi-Fi w podstawowych modelach.

2. ESP32 / ESP8266

- **Budowa:** mikrokontrolery z wbudowanym Wi-Fi i/lub Bluetooth. ESP32 ma dwurdzeniowy procesor i więcej pamięci.
- **Zastosowanie w IoT:** urządzenia podłączone do sieci, sensory, inteligentne domy, sterowanie zdalne.
- **Zalety:**
 - wbudowana łączność sieciowa,
 - mocniejsze CPU i więcej pamięci niż Arduino,
 - obsługa FreeRTOS i wielu języków (C++, MicroPython).
- **Wady:** wyższa złożoność konfiguracji niż Arduino, większe zużycie energii niż prostsze mikrokontrolery.

3. Raspberry Pi (mikrokomputer)

- **Budowa:** pełnoprawny komputer jednopłytkowy z Linuxem, GPIO do sterowania urządzeniami.
- **Zastosowanie w IoT:** analiza danych lokalnie, bramy IoT, bardziej złożone aplikacje wymagające systemu operacyjnego.
- **Zalety:**
 - duża moc obliczeniowa,
 - możliwość użycia języków wysokiego poziomu (Python, Java),
 - obsługa komunikacji sieciowej, USB, kamer itp.
- **Wady:** większe zużycie energii, większy koszt, bardziej skomplikowana konfiguracja w porównaniu do mikrokontrolerów.

Platforma	Typ	Zastosowanie	Zalety	Wady
Arduino	Mikrokontroler	Prototypy, proste czujniki	Prosta, tania, duża społeczność	Mała moc, brak Wi-Fi w podstawowych modelach
ESP32/ ESP8266	Mikrokontroler z Wi-Fi/Bluetooth	Inteligentne urządzenia IoT	Wbudowana łączność, mocniejsze CPU	Złożona konfiguracja, większe zużycie energii
Raspberry Pi	Mikrokomputer	Analiza danych, bramy IoT, aplikacje Linux	Duża moc, pełny OS, wiele języków	Większy koszt, większe zużycie energii

52. Scharakteryzować kryptoanalizę – łamanie szyfrów (przykładowe metody: Brute-force, Ciphertext, Known-Plaintext, Chosen-Plaintext, Chosen-Ciphertext, Meet-in-the-middle)

Jasne — **bardzo prosto i „egzaminowo”**:

Kryptoanaliza

Kryptoanaliza to **łamanie szyfrów**, czyli próba odczytania zaszyfrowanej wiadomości lub klucza **bez jego znajomości**.

Proste metody łamania szyfrów

- **Brute-force**
Sprawdzanie **wszystkich możliwych kluczy**, aż trafi się poprawny.
- **Ciphertext-only**
Atakujący ma **tylko zaszyfrowany tekst** i próbuje go rozszyfrować.
- **Known-Plaintext**
Znany jest **fragment tekstu jawnego i jego szyfrogram**, co ułatwia złamanie szyfru.
- **Chosen-Plaintext**
Atakujący **sam wybiera tekst**, który jest szyfrowany, i analizuje wynik.
- **Chosen-Ciphertext**
Atakujący **wybiera szyfrogram** i sprawdza, co po odszyfrowaniu otrzyma.
- **Meet-in-the-middle**
Atak na **wielokrotne szyfrowanie**, polegający na „spotkaniu się obliczeń pośrodku”, co skraca czas ataku.

Im **więcej informacji** ma atakujący lub im większą ma kontrolę nad danymi, tym **łatwiej złamać szyfr**.

53. Omówić wykorzystanie funkcji haszujących w kryptografii

Funkcja haszująca przekształca dane o dowolnej długości w **skrót (hash) o stałej długości** i jest jednokierunkowa (nie da się odtworzyć danych wejściowych).

- **Przechowywanie haseł**
Zamiast haseł zapisuje się ich **hashe** (np. z użyciem soli), co chroni użytkowników w razie wycieku bazy danych.
- **Sprawdzanie integralności danych**
Hash pozwala wykryć, czy dane **nie zostały zmienione** (np. pobieranie plików).
- **Podpisy cyfrowe**
Najpierw haszowana jest wiadomość, a dopiero hash jest podpisywany kluczem prywatnym – szybciej i bezpieczniej.
- **Uwierzytelnianie i protokoły bezpieczeństwa**
Funkcje haszujące są używane w protokołach takich jak **TLS, IPsec, blockchain**.

- **Skróty w strukturach danych**

Hashowanie wykorzystywane jest w tablicach mieszających i systemach kontroli wersji (np. Git).

Właściwości bezpiecznej funkcji haszującej

- odporność na kolizje (trudno znaleźć dwa różne wejścia z tym samym hashem),
- odporność na odwrócenie (jednokierunkowość),
- wrażliwość na zmianę danych (efekt lawiny).

54. Omówić różnicę pomiędzy serwerem iteracyjnym a współbieżnym

1. Serwer iteracyjny

- **Działanie:** obsługuje **jedno żądanie na raz**.
- **Proces:** klient łączy się → serwer przetwarza żądanie → wysyła odpowiedź → przechodzi do następnego klienta.
- **Zalety:** prosty, łatwy do implementacji, małe zużycie zasobów.
- **Wady:** wolny przy wielu klientach, każdy kolejny klient musi czekać.

2. Serwer współbieżny

- **Działanie:** obsługuje **wiele żądań jednocześnie**.
- **Proces:** dla każdego klienta tworzy **wątek, proces lub task** → żądania są przetwarzane równolegle.
- **Zalety:** szybka obsługa wielu klientów, lepsze wykorzystanie CPU.
- **Wady:** większa złożoność implementacji, wyższe zużycie pamięci i ryzyko problemów współbieżności.

Cecha	Serwer iteracyjny	Serwer współbieżny
Liczba klientów	1 w danym czasie	wielu jednocześnie
Szybkość reakcji przy dużej liczbie klientów	wolna	szybka
Złożoność implementacji	niska	wysoka
Wykorzystanie zasobów	niskie	wyższe

55. Omówić wielopoziomową architekturę sieci przełączanych

Wielopoziomowa architektura sieci przełączanych to **podział sieci LAN na warstwy logiczne**, aby zwiększyć wydajność, bezpieczeństwo i skalowalność. Najczęściej stosuje się **trzy warstwy** (model Cisco):

2. Warstwy architektury

1. Warstwa dostępu (Access Layer)

- Punkt, w którym **urządzenia końcowe (komputery, drukarki, IoT)** łączą się z siecią.
- Funkcje: przełączanie ramek, autoryzacja użytkowników (802.1X), VLAN-y.

2. Warstwa dystrybucji (Distribution Layer)

- Łączy warstwę dostępu z rdzeniem sieci.

- Funkcje: routing między VLAN-ami, kontrola dostępu, agregacja ruchu, polityki bezpieczeństwa.

3. Warstwa rdzeniowa (Core Layer)

- Szybki transport dużej ilości danych między różnymi częściami sieci.
- Funkcje: wysoka przepustowość, minimalne opóźnienia, niezawodność.

3. Zalety wielopoziomowej architektury

- **Skalowalność:** łatwe dodawanie nowych segmentów sieci.
- **Wydaźność:** ruch lokalny pozostaje w warstwie dostępu, nie obciąża rdzenia.
- **Bezpieczeństwo i zarządzanie:** polityki bezpieczeństwa i routing skoncentrowane w warstwie dystrybucji.
- **Redundancja:** w warstwie rdzeniowej można stosować redundantne połączenia dla niezawodności.

56. Omówić problematykę pętli switchingu

1. Co to jest pętla switchingowa?

- Pętla powstaje, gdy w sieci przełączanej istnieje więcej niż jedna ścieżka między switchami.
- Ramki Ethernet mogą wtedy krążyć w nieskończoność, co prowadzi do przeciążenia sieci.

2. Dlaczego pętla jest problemem

- **Nieograniczone powielanie ramek** → zalewanie łączy (broadcast storm).
- **Zajęcie całego pasma** → opóźnienia i utrata pakietów.
- **Nieprawidłowe działanie MAC table** → switche stale aktualizują tablice adresów, co powoduje błędy w przełączaniu.

Pętla switchingu to problem wynikający z nadmiarowych ścieżek między switchami.

Powoduje **broadcast storm i przeciążenie sieci**.

Rozwiązaniem są protokoły takie jak **STP/RSTP**, które automatycznie blokują nadmiarowe połączenia i zapewniają bezpieczną topologię.

57. Przedstawić koncepcje tunelowania i wirtualizacji sieci na przykładzie protokołu GRE

Tunelowanie i wirtualizacja sieci – GRE

- **Tunelowanie:** przesyłanie pakietów jednej sieci w pakietach innej sieci („pakiet w pakiecie”).
- **Wirtualizacja sieci:** tworzenie **logicznej sieci** niezależnej od fizycznej infrastruktury.
- **GRE (Generic Routing Encapsulation):**
 - enkapsuluje pakiety (IPv4, IPv6, multicast) w pakiety IP,
 - umożliwia łączenie oddziałów firmy w jedną wirtualną sieć,
 - **sam nie szyfruje ani nie uwierzytelnia** danych.
- **Jak działa:**
 - Router „opakowuje” pakiet w nagłówek GRE,
 - Pakiet przechodzi przez Internet,
 - Drugi router usuwa nagłówek i przekazuje oryginalny pakiet.

- **Zastosowanie:** wirtualne połączenia między sieciami, przenoszenie multicastu, podstawa dla VPN (GRE + IPsec).

58. Omówić rolę ACL w zarządzaniu sieciami i poprawie bezpieczeństwa

ACL (Access Control List) to lista reguł stosowanych w routerach i switchach, która **określa, jakie pakiety mogą przechodzić przez sieć, a jakie mają być blokowane.**

2. Rola ACL

- **Kontrola dostępu:** pozwala przydzielać dostęp do zasobów tylko wybranym użytkownikom lub urządzeniom.
- **Filtrowanie ruchu:** blokuje niepożądany ruch (np. ataki, nieautoryzowane protokoły).
- **Poprawa bezpieczeństwa:** ogranicza możliwość włamań i nieautoryzowanego przesyłu danych.
- **Zarządzanie ruchem:** można definiować priorytety i kierować ruch zgodnie z polityką firmy.

3. Typowe zastosowania

- Blokowanie adresów IP lub podsieci, które nie powinny mieć dostępu.
- Zezwalanie tylko na określone protokoły (np. HTTP, SSH).
- Ograniczanie ruchu między VLAN-ami lub segmentami sieci.

ACL to **podstawowe narzędzie bezpieczeństwa w sieciach**, pozwalające **kontrolować, filtrować i zarządzać ruchem**, minimalizując ryzyko nieautoryzowanego dostępu.

59. Omówić znaczenie lokalnych i globalnych cech charakterystycznych obrazów w zadaniach detekcji, lokalizacji i klasyfikacji obiektów – rozpoznawania obrazów.

1. Cechy obrazów w rozpoznawaniu obiektów

- **Cechy lokalne** – informacje pochodzące z **małych fragmentów obrazu** (np. punkty, krawędzie, rogi).
- **Cechy globalne** – informacje o **całym obrazie lub dużych jego fragmentach** (np. kolor, tekstura, kształt obiektu).

2. Znaczenie cech lokalnych

- **Detekcja obiektów:** pozwalają wykryć **lokalne szczegóły** charakterystyczne dla obiektu (np. oczy w twarzy).
- **Odporność na zmiany:** działają dobrze przy obrotach, skalowaniu czy częściowym zasłonięciu obiektu.
- **Przykłady:** SIFT, SURF, ORB – wykrywają charakterystyczne punkty i ich otoczenie.

3. Znaczenie cech globalnych

- **Klasyfikacja obiektów:** pozwalają określić, **co jest na obrazie** na podstawie ogólnego wyglądu.
- **Lokalizacja obiektu:** mogą pomóc w szybkim zawężeniu obszaru zainteresowania.
- **Przykłady:** histogramy kolorów, cechy tekstury (GLCM), momenty Hu, sieci konwolucyjne (CNN) – cechy wyciągane z całego obrazu.

Typ cechy	Zastosowanie	Zalety
Lokalna	Detekcja i rozpoznawanie fragmentów obiektu	Odporność na przesunięcia, skalowanie, częściowe zakrycie
Globalna	Klasyfikacja, wstępna lokalizacja	Informacja o całym obrazie, szybkie określenie kategorii obiektu

60. Zaprezentować metody rozpoznawania mówcy – deskryptory głosu.

Okej, oto proste i zrozumiałe omówienie **rozpoznawania mówcy i deskryptorów głosu**.

1. Rozpoznawanie mówcy

- Celem jest **identyfikacja lub weryfikacja osoby** na podstawie jej głosu.
- Stosuje się w: systemach bezpieczeństwa, bankowości telefonicznej, asystentach głosowych.

2. Metody rozpoznawania mówcy

1. Analiza cech akustycznych

- Ekstrakcja **cech mowy**, które są charakterystyczne dla danej osoby (barwa, ton, tempo).

2. Statystyczne modele mówcy

- Modele takie jak **GMM (Gaussian Mixture Models)** lub **HMM (Hidden Markov Models)** opisują prawdopodobieństwo występowania cech danej osoby.

3. Metody oparte na uczeniu maszynowym

- Sieci neuronowe, np. **długie sieci rekurencyjne (LSTM)** lub CNN, które uczą się charakterystycznych wzorców głosu.

3. Deskryptory głosu

- **Deskryptor** = zestaw liczbowych cech reprezentujących cechy mówcy.
- **Przykłady deskryptorów:**
 - **MFCC (Mel-Frequency Cepstral Coefficients)** – najpopularniejsze, opisują kształt widma mowy w sposób podobny do ludzkiego słuchu.
 - **LPCC (Linear Predictive Cepstral Coefficients)** – przewidują próbki mowy na podstawie wcześniejszych próbek.
 - **PLP (Perceptual Linear Prediction)** – uwzględniają percepcję ludzkiego słuchu.

4. Podsumowanie

- **Rozpoznawanie mówcy** = identyfikacja osoby po głosie.
- **Metody:** analiza akustyczna, statystyczna, uczenie maszynowe.
- **Deskryptory głosu** (MFCC, LPCC, PLP) zamieniają cechy mowy na liczby, które służą do porównania i klasyfikacji mówców.